Experiment -10

Python Program To Implement A* Search Algorithm

Code:

```
import heapq

def a_star(graph, start, goal, heuristic):
    open_list = [(heuristic[start], 0, start, [start])]
    best_cost = {start: 0}

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        if g > best_cost[current]:
            continue

        if current == goal:
            return path, g

        for neighbor, cost in graph[current]:
            new_g = g + cost

            if new_g < best_cost.get(neighbor, float('inf')):
                best_cost[neighbor] = new_g
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    open_list,
```

```
                (new_f, new_g, neighbor, path + [neighbor])
            )

    return None, float('inf')


graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 4)],
    'E': [('G', 2)],
    'F': [('G', 2)],
    'G': []
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 4,
    'E': 2,
    'F': 2,
    'G': 0
}

start = 'A'
```

```
goal = 'G'

path, cost = a_star(graph, start, goal, heuristic)

if path:
    print("Shortest Path:", " -> ".join(path))
    print("Total Cost:", cost)
else:
    print("No path found")
```

```python
import heapq

def a_star(graph, start, goal, heuristic):
    open_list = [(heuristic[start], 0, start, [start])]
    best_cost = {start: 0}

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        if g > best_cost[current]:
            continue

        if current == goal:
            return path, g

        for neighbor, cost in graph[current]:
            new_g = g + cost

            if new_g < best_cost.get(neighbor, float('inf')):
                best_cost[neighbor] = new_g
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    open_list,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    return None, float('inf')

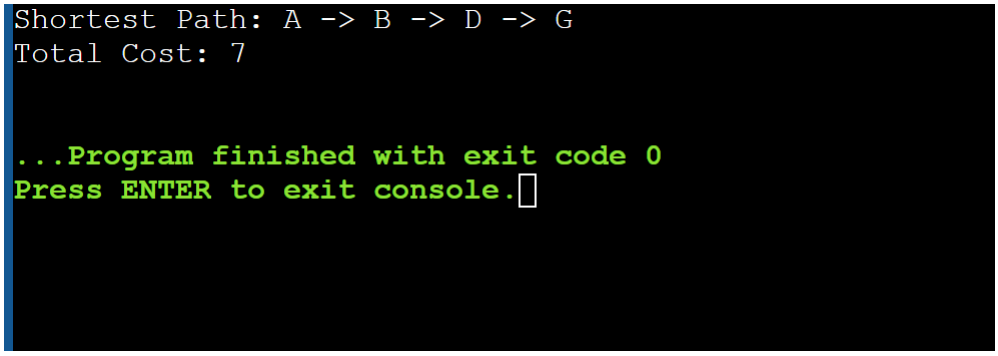

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 4)],
    'E': [('G', 2)],
    'F': [('G', 2)],
    'G': []
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 4,
    'E': 2,
    'F': 2,
    'G': 0
}

start = 'A'
goal = 'G'

path, cost = a_star(graph, start, goal, heuristic)

if path:
    print("Shortest Path:", " -> ".join(path))
    print("Total Cost:", cost)
else:
    print("No path found")
```


Shortest Path: A -> B -> D -> G
Total Cost: 7
...Program finished with exit code 0
Press ENTER to exit console.